

Task 2 and 3: (ANN.py , optimizers.py)

1) Problem setup

- Network: fully-connected $2 \rightarrow 5 \rightarrow 2$ with sigmoid activations (same as the paper's code).
- Dataset: Higham 2-class toy set (10 points).
- Loss used by our new optimizers: average squared error

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \frac{1}{2} \|f_{\theta}(x_i) - y_i\|^2.$$

- Baseline optimizer (paper): per-sample **SGD with fixed learning rate** (“original SGD”).
We also tested:
 - **BatchGD (fixed η)** – full-batch steepest descent with constant step.
 - **LineSearchGD (Armijo)** – full-batch steepest descent with backtracking line search.
 - **BFGS (weak Wolfe)** – full-batch quasi-Newton with weak Wolfe line search.
 - **SciPy BFGS** – `scipy.optimize.minimize(..., method="BFGS")` on our packed parameter vector and full-batch loss/grad.

The figures in the Folder “Task 2+3 Result Screenshots” are the basis of the empirical comparison.

2) Methods & mathematics (what each algorithm is doing)

Original SGD (paper)

At epoch (t), sample index (k_t) is drawn (shuffled, no replacement), and the update is

$$\theta_{t+1} = \theta_t - \eta g(\theta_t; x_{k_t}, y_{k_t}), \quad g(\theta; x, y) = \nabla_{\theta} \left[\frac{1}{2} \|f_{\theta}(x) - y\|^2 \right].$$

- **Descent direction:** the negative stochastic gradient $(-g)$.
- **Step length:** constant learning rate η (no line search).
- **Properties:** simple, low memory, noisy but explores; convergence can be slow and sensitive to chosen learning rate η .

BatchGD (fixed η)

Same direction as above but with the full-batch gradient:

$$g(\theta) = \nabla L(\theta).$$

- **Pros:** stable; **Cons:** if η is conservative it's slow; if too large it oscillates.

LineSearchGD (Armijo)

Direction is steepest descent ($p_k = -g_k$). Step length (α_k) chosen by **Armijo backtracking**:

$$L(\theta_k + \alpha p_k) \leq L(\theta_k) + c_1 \alpha g_k^\top p_k, \quad 0 < c_1 < 1.$$

If not satisfied, shrink $\alpha \leftarrow \rho \alpha$ with $\rho \in (0, 1)$ (optionally using quadratic/cubic interpolation instead of pure halving).

- **Effect:** automatically avoids steps that don't decrease the loss sufficiently; removes sensitivity to a global learning rate.

BFGS (weak Wolfe)

Search direction uses an **inverse-Hessian approximation** (H_k):

$$p_k = -H_k g_k, \quad g_k = \nabla L(\theta_k).$$

Step length chosen by **weak Wolfe**:

$$\underbrace{L(\theta_k + \alpha p_k) \leq L(\theta_k) + c_1 \alpha g_k^\top p_k}_{\text{sufficient decrease}} \quad \text{and} \quad \underbrace{\nabla L(\theta_k + \alpha p_k)^\top p_k \geq c_2 g_k^\top p_k}_{\text{curvature}},$$

with $0 < c_1 < c_2 < 1$ (we used $c_1 \approx 10^{-4}$, $c_2 \in [0.5, 0.9]$)

After accepting (α_k) , update (H_k) by the BFGS formula with

$$s_k = \theta_{k+1} - \theta_k, \quad y_k = g_{k+1} - g_k:$$

$$H_{k+1} = (I - \rho_k s_k y_k^\top) H_k (I - \rho_k y_k s_k^\top) + \rho_k s_k s_k^\top, \quad \rho_k = \frac{1}{y_k^\top s_k}.$$

- **Effect:** the preconditioned direction $(-H_k g_k)$ is much closer to Newton's step; with Wolfe, (H_k) stays positive definite and directions remain descent. This typically yields **superlinear** convergence near a minimizer.

SciPy BFGS

Same mathematics as BFGS above, but implemented/engineered by SciPy with robust line search and tested safeguards. It operates on our packed parameter vector and full-batch loss/grad.

3) Empirical results (what the plots show)

Learning curves (log-scale “Cost” vs Step/Epoch)

Across your runs:

- **Original SGD (paper):**
Characteristic *staircase* decrease—as samples change, the loss jitters then drops in bursts. With enough epochs it reaches low loss (e.g., (10^{-3}) – (10^{-2})) range in your plots), but **slower** than line-search methods.
- **BatchGD (fixed η):**
Slowest. With a conservative η (chosen to avoid divergence), descent is smooth but shallow; the curve flattens at a relatively high loss (e.g., (10^{-1}) – (10^{-2})).
- **LineSearchGD (Armijo):**
Clearly faster than both fixed- η methods. Early loss drops by 1–2 orders of magnitude quickly and then continues descending steadily. Removes the need to hand-tune a global η .
- **BFGS (weak Wolfe):**
Fastest by far. In your runs it drops to extremely small losses (down to (10^{-9}) – (10^{-11})) within a tiny number of iterations and then stays there. This is the expected quasi-Newton behavior on a small, well-conditioned problem with a smooth loss.

- **SciPy BFGS:**
Very rapid early decrease (often faster initially than LineSearchGD, slower than our in-repo BFGS because SciPy's default stopping and line-search tolerances are conservative). It typically plateaus around (10^{-4}) – (10^{-5}) in your runs—still far better than fixed- η methods and comparable to LineSearchGD. Tightening tolerances or allowing more iterations would push it lower.

Decision boundaries

- All methods learn a **reasonable separating curve** for the two clusters.
- **Original SGD** and **LineSearchGD** show lightly **curved** boundaries (sigmoid network + hidden units).
- **BatchGD** can be a bit **under-fit** (boundary more linear sometimes) due to slow progress. It learns almost the same curved boundary as other methods if learning rate is picked properly.
- **BFGS (weak Wolfe)** and **SciPy BFGS** yield clean, stable boundaries consistent with the low training loss. For our **BFGS (weak Wolfe)** implementation, this is only true if hyperparameters for line search and number of steps are picked carefully (see below for values that were found to work best for this homework assignment)

Variability across runs in the **original SGD** boundary is due to its stochastic per-sample updates and the fixed learning rate; line search largely removes that sensitivity.

4) Why these outcomes make sense

- **A constant learning rate** trades simplicity for sensitivity: too small $\rightarrow$ slow; too big $\rightarrow$ oscillation or divergence. This is why BatchGD is slow and SGD needs many passes to reach the same loss as line-search methods.
- **Armijo line search** adapts the step to local curvature along $(-g)$: big steps on flat regions, small steps near steep zones—hence consistent speedup over fixed- η GD/SGD without extra math.
- **BFGS** uses curvature information accumulated in (H_k) . The direction $(-H_k g_k)$ **preconditions** the gradient, undoing ill-conditioning, so the line search finds large, safe steps. On small smooth problems like this, it often looks “unfairly” fast and that’s normal quasi-Newton behavior.
- **SciPy BFGS** is robust and general-purpose; with defaults it may accept slightly smaller steps and stop a bit earlier than our hand-tuned version. Turning the knobs (`gtol`, `maxiter`, line-search tolerance) typically closes the small gap.

5) Short algorithm summaries

1. Original SGD (fixed learning rate, per-sample)

For a single training example (x_{k_t}, y_{k_t}) :

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \left(\frac{1}{2} \|f_{\theta}(x_{k_t}) - y_{k_t}\|^2 \right)$$

- Descent direction: the negative stochastic gradient.
- Step size: constant η .

2. Batch Gradient Descent (BatchGD)

With full dataset $\{(x_i, y_i)\}_{i=1}^N$:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \frac{1}{2} \|f_{\theta}(x_i) - y_i\|^2$$

Update rule:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t)$$

- Gradient averaged over all samples.
- Fixed step size η .

3. Line Search Gradient Descent (Armijo)

Direction:

$$p_t = -\nabla L(\theta_t)$$

Armijo condition:

$$L(\theta_t + \alpha p_t) \leq L(\theta_t) + c_1 \alpha \nabla L(\theta_t)^{\top} p_t, \quad 0 < c_1 < 1$$

Update:

$$\theta_{t+1} = \theta_t + \alpha p_t$$

4. Line Search Stochastic Gradient Descent (LS-SGD)

Per-sample loss:

$$\ell(\theta; x, y) = \frac{1}{2} \|f_{\theta}(x) - y\|^2$$

Update rule:

$$\theta_{t+1} = \theta_t - \alpha_t \nabla_{\theta} \ell(\theta_t; x_{k_t}, y_{k_t})$$

where α_t is chosen by Armijo line search.

5. BFGS with Weak Wolfe Line Search

Direction:

$$p_t = -H_t \nabla L(\theta_t)$$

Weak Wolfe conditions:

$$L(\theta_t + \alpha p_t) \leq L(\theta_t) + c_1 \alpha \nabla L(\theta_t)^\top p_t$$

$$\nabla L(\theta_t + \alpha p_t)^\top p_t \geq c_2 \nabla L(\theta_t)^\top p_t$$

with $0 < c_1 < c_2 < 1$.

Parameter update:

$$\theta_{t+1} = \theta_t + \alpha p_t$$

BFGS Hessian update:

$$H_{t+1} = (I - \rho_t s_t y_t^\top) H_t (I - \rho_t y_t s_t^\top) + \rho_t s_t s_t^\top$$

where

$$s_t = \theta_{t+1} - \theta_t, \quad y_t = \nabla L(\theta_{t+1}) - \nabla L(\theta_t), \quad \rho_t = \frac{1}{y_t^\top s_t}$$

6. SciPy BFGS

Uses the exact same formulas as above (BFGS direction, Wolfe line search, BFGS Hessian update), but with SciPy's robust line search implementation and stopping criteria.

6) Head-to-head conclusions

Speed & final loss (best → worst) on this problem:

1. **BFGS (weak Wolfe)** – orders-of-magnitude lower loss with very few steps.
2. **SciPy BFGS ≈ LineSearchGD** – both rapidly reach low loss; SciPy often stops near (10^{-5}) by default.
3. **Original SGD** – eventually good, but slower; depends on random order and η .
4. **BatchGD (fixed η)** – slowest and can under-fit within a fixed budget.

Stability / tuning effort

- Least tuning: **LineSearchGD** and **BFGS** (line search chooses the step).
- Most tuning: **Original SGD** and **BatchGD** (η sensitive).

Boundary quality

- All non-trivial methods learn a similar (curved) separator.
- Under the same budget, **BFGS** and **LineSearchGD** produce the **cleanest** decision boundaries if parameters for line search and number of steps are picked properly as discussed earlier.

8) Overall Conclusion

Line search (both in GD and in BFGS) provides dramatic improvements over fixed learning rate methods by adapting the step length to the local curvature.

BFGS with weak Wolfe conditions achieves superlinear convergence and the lowest loss, while SciPy's BFGS is robust and competitive, though defaults are conservative.

For small problems like the Higham dataset, quasi-Newton methods clearly dominate, but SGD remains valuable for larger-scale problems where full-batch Hessian approximations are too costly.

Both BFGS (weak Wolfe) and SciPy BFGS yield clean, stable classification boundaries consistent with the low training loss.

However, if the maximum number of iterations for the line search or the Wolfe condition tolerances are not chosen carefully, the algorithms may produce unstable results: either linear misclassification boundaries or irregular, overfitted curves.

This highlights the sensitivity of line-search-based methods to their hyperparameters.

In our experiments, the following parameter choices yielded stable and accurate results:

- Weak Wolfe line search parameters:
 - α or $c_1 = 10^{-4}$, β or $c_2 = 0.9$
 - Maximum backtracking iterations: 50
 - initial step size: $\lambda_0 = 1.0$
 - item Safeguards: minimum step length $\lambda_{\min} = 0.1$, maximum step length $\lambda_{\max} = 10$

These settings consistently produced fast convergence and clean classification boundaries.

8) Why not always use BFGS since it has fastest convergence?

- Memory ($O(n^2)$) for (H) (fine here; expensive for very large nets).
- Requires **full-batch** gradients for clean Wolfe satisfaction; with noisy (mini-batch) gradients you need damping/fallbacks.

Task 4: (digimon_train.py , ANN.py , optimizers.py, DigiDB_digimonlist.csv)

For this task I tried extending our comparison of optimizers to a **real-world dataset**: *DigiDB Digimon List*.

The dataset includes each Digimon's stage, type, attribute, memory cost, equip slots, and level-50 stats (HP, SP, Atk, Def, Int, Spd).

I formulated a **binary classification problem**:

- **Label (target)**: whether a Digimon is a *Rookie* (1) or *non-Rookie* (0).
- **Features (inputs)**: we selected two numeric attributes:
 - Lv50 Atk (attack power at level 50)
 - Memory (team cost of the Digimon).

These two dimensions give a clear separation between Rookies and evolved Digimon, making the problem suitable for visualization of decision boundaries.

I tried training the same **2 → 10 → 2 fully-connected network** with sigmoid activations using three methods:

1. **Original SGD (fixed learning rate)**: the baseline method used in the paper.
2. **LineSearch SGD (Armijo)**: stochastic gradient descent where the step size is adaptively chosen by Armijo backtracking line search.
3. **Stochastic mini-batch BFGS with weak Wolfe conditions**: a quasi-Newton stochastic variant.

In **Stochastic BFGS**, instead of computing the gradient over the full dataset, we only use a **mini-batch** of samples:

1. Randomly select a batch $\mathcal{B}_t \subset \{1, \dots, N\}$ of size m .
2. Compute the **stochastic gradient**:

$$g_t = \frac{1}{m} \sum_{i \in \mathcal{B}_t} \nabla \ell(\theta_t; x_i, y_i)$$

3. Compute the search direction using the Hessian approximation:

$$p_t = -H_t g_t$$

4. Perform a **stochastic line search** (Armijo or weak Wolfe) to find α_t , but based only on the mini-batch cost and gradients.
5. Update parameters:

$$\theta_{t+1} = \theta_t + \alpha_t p_t$$

6. Update the inverse Hessian using:

$$s_t = \theta_{t+1} - \theta_t, \quad y_t = g_{t+1} - g_t,$$

where g_{t+1} and g_t are computed on (possibly different) mini-batches.

Bottlenecks and Challenges in Training

While the original Higham toy dataset allowed rapid training and visualization, extending the same framework to the Digimon dataset exposed several performance bottlenecks and implementation issues:

1. Expensive Line Searches

- Both **LineSearch SGD** and **Stochastic BFGS** repeatedly evaluate the loss and gradient inside their line search routines (Armijo or weak Wolfe).
- On the larger Digimon dataset, this means that for each training update, the loss/gradient must be recomputed many times, making each epoch significantly slower than simple fixed-rate SGD.

2. Non-Vectorized Forward Passes

- The `GeneralNetwork.predict()` function processes **one sample at a time**, including during decision boundary plotting.
- This lack of vectorization introduces large overheads when handling datasets with hundreds of Digimon or when plotting thousands of grid points.

3. High Dimensional Parameters

- The Digimon dataset uses numeric features such as Lv50 Atk and Memory, but the chosen network ($2 \rightarrow 10 \rightarrow 2$) has a relatively large number of parameters compared to the small toy dataset.
- When combined with second-order methods like BFGS, the cost of maintaining and updating the Hessian approximation H_t grows as $O(n^2)$, where n is the number of parameters.

While no classification results were obtained on the digimon dataset so far due to the numerous bottlenecks and issues stated earlier, I will be trying different improvements and tweaks to achieve results training the neural network with the different optimizers